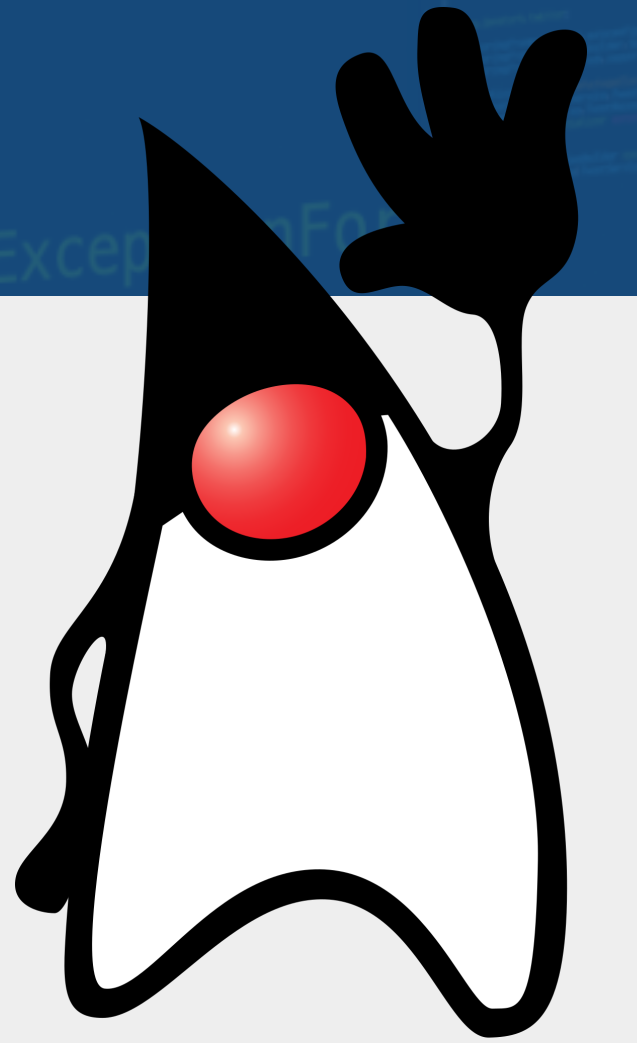


Java Programlama Dili

13. Bölüm

Sınıflar ve Nesneleri

Akın Kaldıroğlu



Amaç



- Bu bölümle birlikte nesne merkezli programlamanın (object-oriented programming) detaylarına girmeye başlayacağız ve ilk olarak sınıflar ve nesnelerini, biraz hatırlatma amaçlı olarak, ele alacağız
- Sonraki iki bölümde nesnelerin durumu ve davranışları incelenecektir
- Sonrasında, kurucular ile this ve static anahtar kelimelerini ele alan bölümler gelecektir
- Bu altı bölümde, en temel sınıf ve nesne yapılarını ele almış olacağız



- **Sınıflar ve Nesneleri**
 - Nesne ve Referans
 - Sınıf mı Nesne mi?
- **Ödevler**
- **Bundan Sonra?**

Sınıflar ve Nesneleri



Sınıf - I



- Sınıf, kendisinden üretilecek nesnelerin kalıbı veya şablonudur
- Aynı sınıftan üretilen nesnelerin tipi aynıdır
- Sınıf, nesnelerinin özelliklerini (attributes, properties) ve davranışlarını (behavior) tanımlar
- Nesnelerin özellikleri, değişkenlerle (variables) tanımlanır
- Nesnelerin davranışları ise metotlarla (methods) tanımlanır

Sınıf - II



- Nesnenin özelliklerinin yani tüm alanlarının bütününe **durum (state)**, davranışlarının yani tüm metotlarının dışarıdan görünümünün bütününe de **arayüz (interface)** denir
- Durum ve arayüz, sınıfta tanımlanır ve sınıfın nesneleri üzerinden ulaşılır
- Sınıf, Java'nın en geniş bloğudur; durum ve arayüz ise sadece sınıfta tanımlanır

Sınıf ve Tip



- Sınıf, yeni bir tip oluşturma ya da tanımlama yapısıdır ama Java'daki tek tip tanımlama yapısı sınıf değildir
- Java'da **soyut sınıflar (abstract classes)** ve **iç sınıflar (nested classes)** gibi farklı yapıda sınıflar olduğu gibi, sınıfın dışında, **sıralama (enum)**, **arayüz (interface)** ve **kayıt (record)** isminde farklı tip oluşturma yapıları da mevcuttur
- Burada 8 ilkel tipi (primitives) göz önüne almıyoruz, sadece nesne tiplerine odaklanıyoruz

Sınıf Tanıtma - I



- Java'da sınıf tanıtmak için **class** anahtar kelimesi kullanılır:

```
<niteleyici>* class <Sınıfİsmi>{  
    <değişken>*  
    <kurucu>*  
    <metot>*  
}
```

- Sınıfın sıfır ya da daha fazla niteleyicisi (modifier) olabilir
- Sınıfın geçerli ve anlamlı bir ismi olmalıdır ve o isim *CamelCase* olarak yazılmalıdır

Sınıf Tanıtma - II



- **Kurucu (yapılandırıcı/constructor)**, nesne yaratılırken çağrılan ve nesnenin ilk durumunu belirlemek (initialization) için kullanılan özel bir metottur
- Zorunlu olmamakla birlikte sınıfın öğeleri, sınıf içinde yukarıdan aşağıya doğru özellikler, kurucular ve metotlar olarak sıralanır

Nesne Yaratmak - I



- Nesne yaratmak ve daha sonra erişmek için bu nesneyi bir referansa atamak, aslen bir değişken tanımlamadır (variable definition)
- Önce, bir değişken olarak referans tanıtılmalı (declaration), sonra da göstereceği nesne oluşturularak (initialization) referansa atanmalıdır (assignment)
- İlkel değişken tanımlamadan en önemli farkı, **new** ve kurucu çağrısıdır

Değişken Tanımlama = Değişken Tanıtma + İlk Değer Atama

Variable Definition = Variable Declaration + Initialization

```
Book bookObject = new Book();
```

Nesne Yaratmak - II



- Nesne yaratmanın detayları şöyledir:
- **Tanıtım (Declaration)**: Yaratılacak nesneyi gösterecek referans tanıtılır
- **Yaratma (Instantiation)**: **new** ile kurucu çağrısı yapmak için kullanılır

```
Book bookObject = new Book();
```

- **Başlangıç duruma getirme (Initialization)**: Kurucu çağrısı ile nesne yaratılır ve başlangıç durumuna getirilir
- **Atama (Assignment)**: Nesnenin adresi referansa atanır

Nesne Yaratmak - II



- **new** anahtar kelimesi, daima kurucu çağrısından önce ve sadece onunla kullanılır
- Java'da birkaç istisna dışında nesneler, daima **new** ve takip eden kurucu çağrısıyla oluşturulur
 - Bunun en bilinen istisnaları **String** ve dizilerdir (arrays)
- Her nesne, bellekte (heap) ayrı bir alana sahiptir, nesne özelliklerinin değerleri, heapte, nesneye ayrılan bellek alanında tutulur

Nesne Yaratmak - IV



- Oluşturulan nesnenin tipi kurucu çağrısıyla belirlenir
- Nesnenin tipi ile referansının tipi aynı olmalıdır

```
Book bookObject = new Book();
```

- Bunu şimdilik bu basitlikte kabul edelim

Nesne Yaratmak - V



- Nesne oluşturmak ve bir referansa atamak, daha önce ele alındığı gibi, tanıma ve ilk değer atamadan oluşur
- Bu iki adım, tek bir yerde yapılabileceği gibi önce referans tanıtmayı bir yerde, daha sonra da nesneyi yaratma, başlangıç durumuna getirme ve atama topluca başka bir yerde yapılabilir

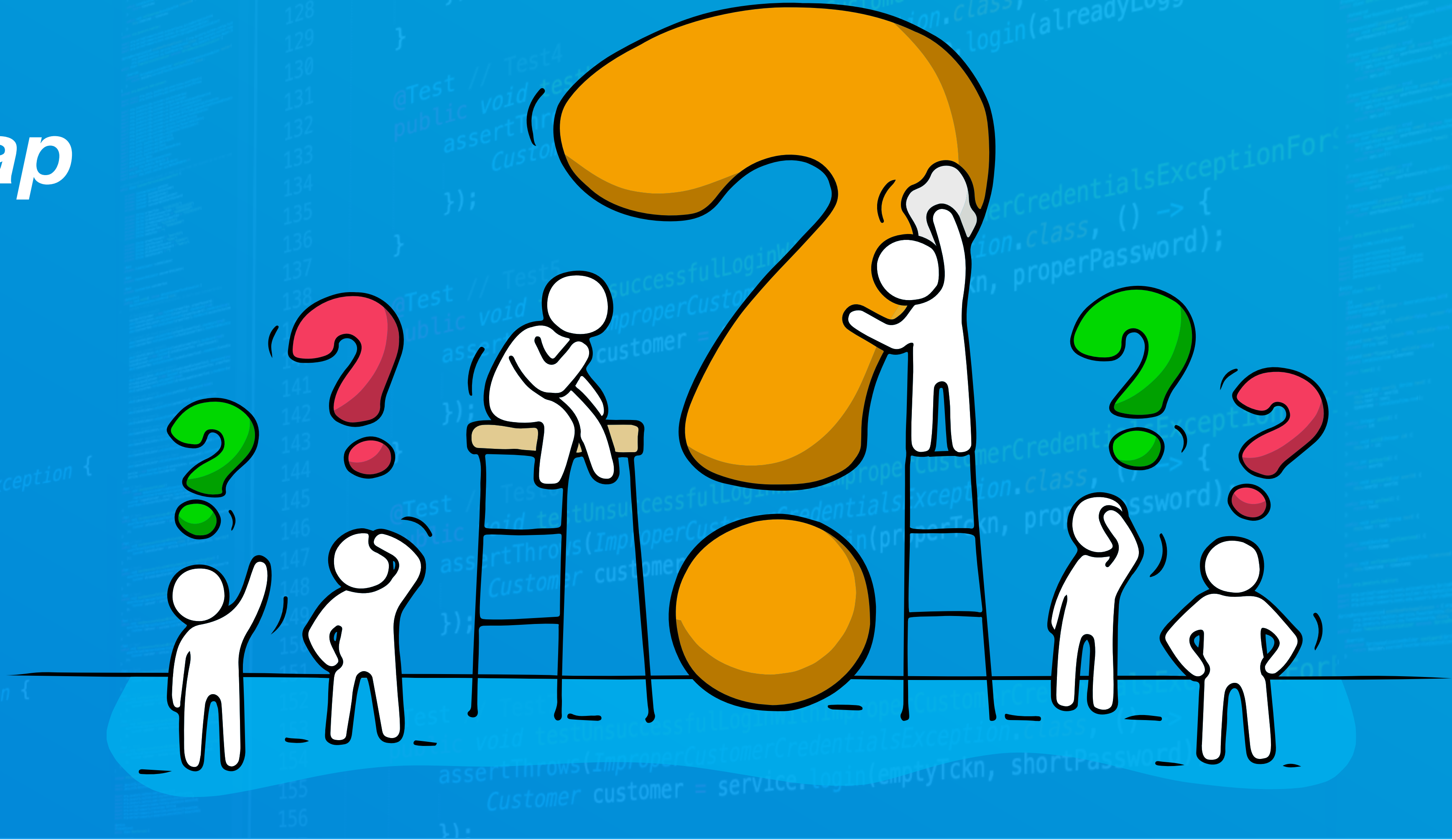
```
Book bookObject;           // Step 1
...
// bookObject isn't assigned anything yet!
...
bookObject = new Book(); // Step 2, 3 & 4
```

Heap



- Nesneler, **heap** adı verilen ve RAM'in JVM tarafından dinamik olarak kullanılan alanında oluşturulur
- Nesnelerin özellikleri olan değişkenler bu alanda yaşar
- Java'da, bu bellek alanına doğrudan ulaşmamız ya da müdahale etmemiz mümkün değildir

Soru ve Cevap Zamanı!





Nesne ve Referans (Hatırlatma)



- Java'da nesnelere sadece **referanslarıyla (reference)** ulaşılır
- Referansa aşağıdaki isimler de verilir:
 - **Tutaç (handle)**
 - **İşaretçi (pointer)**
- Java'daki referanslar, C++'taki pointerların soyut halidir
 - C++'ta pointerlarla nesnelerin bellekteki adreslerine ulaşılabilir, hatta pointer aritmetiği (bellekte gezinmek) yapılabilir

Nesne ve Referans - I



- Nesne ve referans, farklı kavramlardır:
- İkisinin de tipi vardır ve (şu an için) aynıdır
- Nesne, sınıftan türetilen, bir grup özellik ve davranışa sahip olan, heap isimli bellek alanında yaşayan yapıdır
- Referans ise, nesneye ulaşmamızı sağlayan ve stack veya heapte bulunan bir değişkendir
- **Soru:** Bir referans ne zaman stackte ne zaman heapte bulunur?

Nesne ve Referans - II



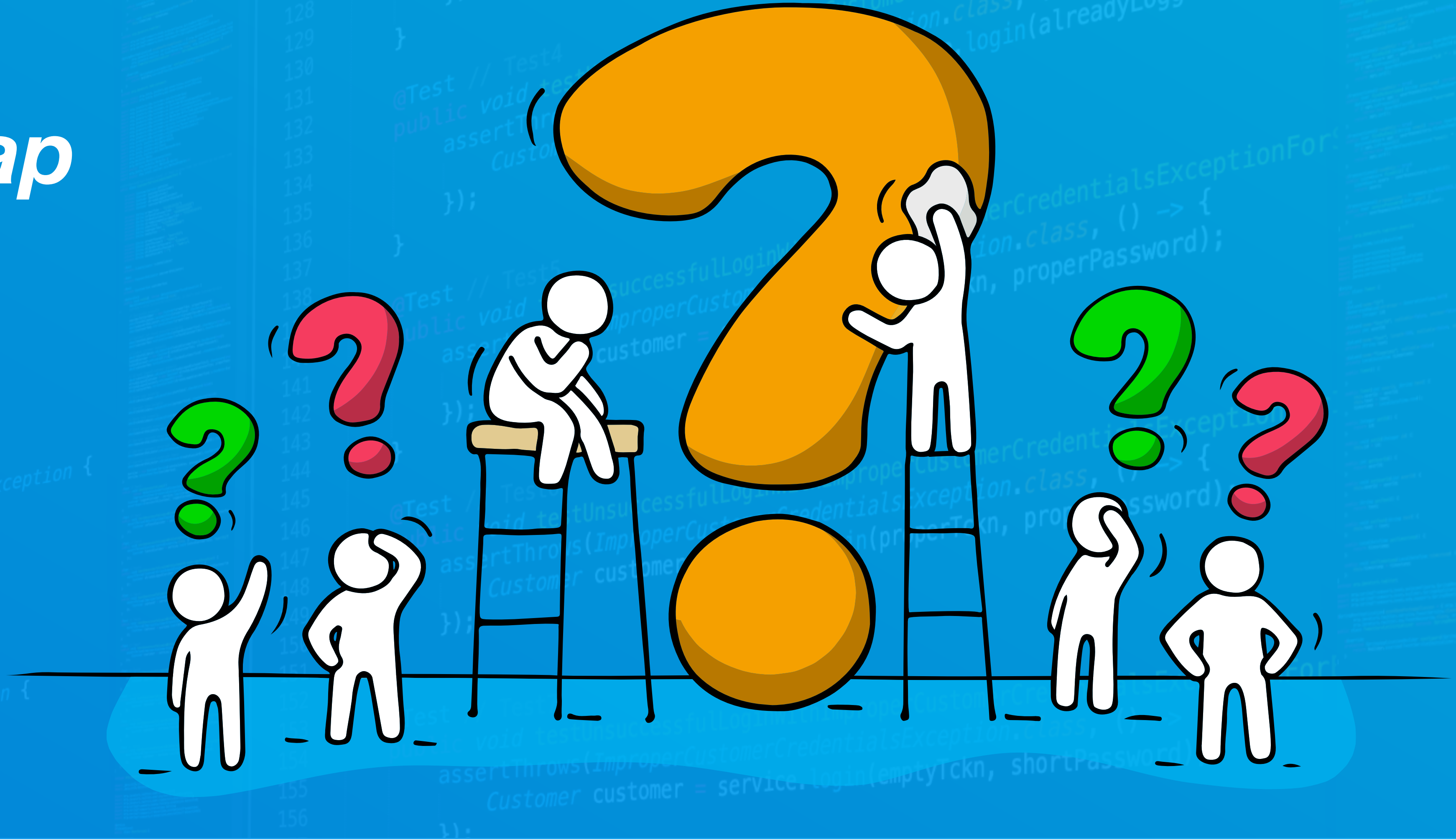
- Bir referans, zamanın farklı anlarında, kendi tipinden olduğu müddetçe, değişik nesneleri gösterebilir
- Fakat bir referans bir anda sadece tek bir nesneyi gösterir
- Ama bir nesneye aynı anda birden fazla referans olabilir
 - Yani pek çok referans aynı anda aynı nesneyi gösterebilir
- Hiçbir nesneyi göstermediği durumda referansın `null` olduğu söylenir; yani referansın değeri `null`'dir

Book & Test



- Book & Test

Soru ve Cevap Zamanı!





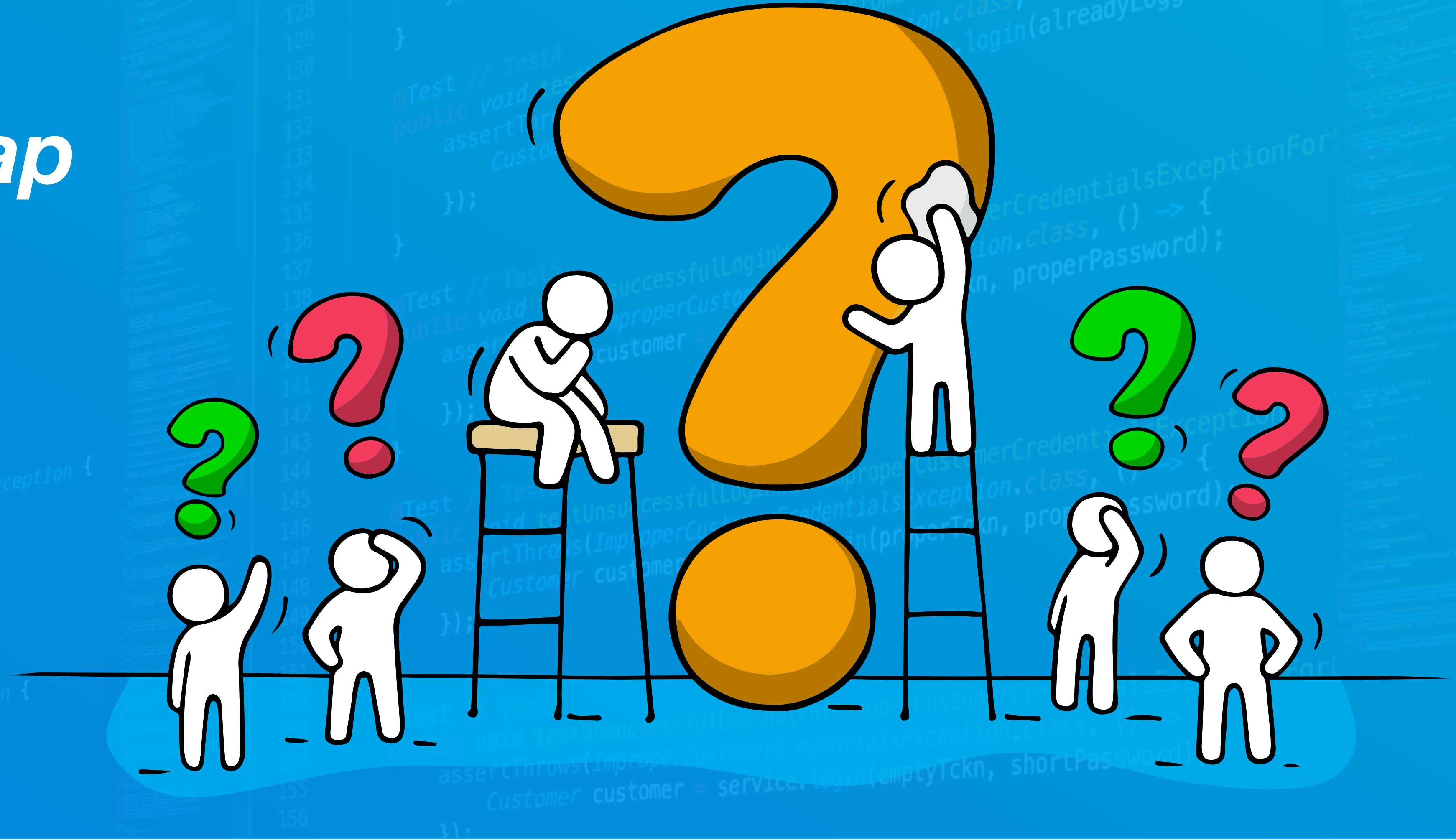
Sınıf mı Nesne mi?

Sınıf mı Nesne mi?



- Sizin de farkettiğiniz gibi *sınıf* dememiz gereken durumlarda bile *nesne* terimini kullanmamız çok sık rastlanan bir durumdur
- Hatta sıklıkla nesne derken aslında referansı kastettiğimiz de olur
- Dolayısıyla *nesne* kelimesi ile ne zaman nesneyi ne zaman sınıfı kastettiğimiz cümlelerin gelişinden anlaşılır
- İlk başta karışık gibi görünse de gittikçe zihniniz buna alışacaktır

Soru ve Cevap Zamanı!



Ödevler





1. Sevdiğiniz bir şarkıyı temsil eden **Song** isminde bir sınıf yazın. Sınıfın yalnızca alanları olsun, metodu olmasın. En az bir **String**, bir **int** ve bir **boolean** alanı bulunsun.
 - a. **SongTest** isminde, **main** metodu olan bir sınıf yazın ve içinde bir **Song** nesnesini tek satırda oluşturun
 - b. İkinci bir **Song** nesnesini, referansı bir yerde tanıtip nesneyi başka bir yerde oluşturacak şekilde iki adımda yaratın
 - c. İki adımın arasında, henüz nesne oluşturulmamışken referansı ekrana basmayı deneyin. Ne oluyor? Sizce neden?



2. **SongTest** sınıfının main metodunda bir **Song** nesnesi oluşturup **song1** referansına atayın. Sonra **Song song2 = song1;** satırını ekleyin.
- a. **song2** üzerinden alanlardan birinin değerini değiştirin
 - b. Aynı alanı **song1** üzerinden ekrana basın
 - c. Gördüğünüz sonucu, ortada kaç nesne ve kaç referans olduğuyula açıklayın



3. Aşağıdaki kodu inceleyin:

- a. Her satırdan sonra kaç Song nesnesi ve kaç referans olduğunu yazın
- b. Son satırdan sonra s1 hangi nesneyi gösteriyor? s2 neyi gösteriyor?
- c. İlk satırda oluşturulan nesneye üçüncü satırdan sonra hâlâ ulaşılabilir mi?

```
Song s1 = new Song();  
Song s2 = new Song();  
s1 = s2;  
s2 = null;
```



4. Aşağıdaki cümlelerin her birinde nesne kelimesiyle gerçekte neyin kastedildiğini belirtin: sınıf mı, nesne mi, yoksa referans mı?
- a. "Bu programda üç tane Song nesnesi var."
 - b. "Song nesnesinin bir title alanı vardır."
 - c. "song1 nesnesinin title alanını değiştirdim."
 - d. "Bu nesneyi null yaptım."

Bundan Sonra?



Bundan Sonra?



- Bundan sonra nesne merkezli programlamada ilerlemeye devam edeceğiz
- Bir sonraki konumuz, nesnelerin durumu hakkındadır

Bölüm Sonu Soru ve Cevap Zamanı!

